

Първо ще дефинираме Абстрактен тип данни като модел който е зададен само с операциите си и техните свойства

Структура от данни е начин за организиране на даден вид данни в паметта.

Един и същ АТД може да се реализира с различни структури от данни и различни свойства

- АТД списък е линейна структура от данни, която представя крайна редица от елементи в която всеки (без последния) има следващ.

Едносвързан списък - всеки възел носи указател към следващия. Последният носи nullptr като следващ или указател към първия (циклически списък)

Двусвързан списък - всеки възел носи указател previous за възела преди него и указател next за възела след него. Първият има previous=nullptr, а последният има next=nullptr, но може и previous на първия да сочи към последния, а next на последния да сочи към първия (циклически списък)

Свойства

	Едносвързан	Двусвързан
insertFirst	$O(1)$	$O(1)$
insertLast	$O(1)$ (*)	$O(1)$ (*)
removeFirst	$O(1)$	$O(1)$
find(x)	$O(n)$	$O(n)$
insertAfter	$O(n)$ (**)	$O(n)$ (**)
insertBefore	$O(n)$ (***)	$O(1)$ (***)

(*) Само ако имаме указател към последния елемент иначе е $O(n)$

(**) Ако имаме указател към възела тогава е $O(1)$

(***) При даден възел

```

class LinkedList {
    struct Node {
        int data;
        Node* next;
        Node(int data, Node* next): data(data), next(next) {}
    };
    Node* First;
    Node* last;
    void deallocate() {
        while(First) {
            Node* current = First;
            First = First->next;
            delete current;
        }
        last = nullptr;
    }
    void copy(const LinkedList& other) { // can't use deallocate directly as private
        First = nullptr;
        last = nullptr;
        for(Node* current = other->First; current != nullptr; current = current->next) {
            insertLast(current->data);
        }
    }
}

```

public:

LinkedList(): first(nullptr), last(nullptr) {}

LinkedList(const LinkedList& other): first(nullptr), last(nullptr) {

 *this = other;

}

LinkedList& operator=(const LinkedList& other) {

 if (&this != &other) {

 deallocate();

 copy(other);

 }

 return *this;

}

~LinkedList() {

 deallocate();

}

void insertLast(int value) {

 Node* newNode = new Node(value);

 if (last != nullptr) {

 last->next = newNode;

 } else {

 first = newNode;

 }

 last = newNode;

}

void insertFirst(int value) {

 first = new Node(value, first);

 if (last == nullptr) {

 last = first;

 }

}

```

void removeFirst() {
    if (isEmpty()) {
        return;
    }
    Node* nextFirst = first->next;
    delete first;
    first = nextFirst;
    if (first == nullptr) {
        last = nullptr;
    }
}

```

```

Node* find (int x) {
    Node* current = first;
    while (current) {
        if (current->value == x) {
            return current;
        }
        current = current->next;
    }
    return nullptr;
}

```

```

void insert After (Node* node, int value) {
    if (node == nullptr) {
        return;
    }
    Node* newNode = new Node (value, node->next);
    node->next = newNode;
    if (node == last) {
        last = newNode;
    }
}

```

```

}

```



```

bool isEmpty() const {
    return First == nullptr;
}

Node* getFirst() const {
    return First;
}

```



• АТД стек е линейна структура с LIFO (Last in - First out) организация
 Операции: `push(x)`, `pop()`, `peek()`, `empty()`

Реализации:

• статична - масив с фиксиран размер със сложност на операциите:

`push(x)` - $O(1)$

`pop()` - $O(1)$

`peek()` - $O(1)$

`empty()` - $O(1)$

Има добра локалност, но е с ограничен размер

• динамична - динамичен масив. При запълване се задава нов масив с по-голям размер (например двоен), но това прави `push(x)` в най-лошия случай да е $O(n)$, но амортизирано е $O(1)$. Ако увеличаваме размера на масива k -орно при всяко препълване броят пъти в които имаме скоп `push(x)` расте логаритмично. Динамичната реализация също има добра локалност и неограничен размер (стиса да имаме достатъчно място), но `push(x)` е скоп при препълване

• свързана - верига от възли с указател към следващия. `push(x)`, `pop()`, `empty()` и `peek()` са с времева сложност $O(1)$, но нямаме постоянност, защото възлите не са последователно в паметта, а са разпрестатни из мемор-та. Следователно при `push(x)`, `pop()` се извършва `new/delete`.

```
class Stack {
    struct Node {
        int value;
        Node* next;
        Node(int value, Node* next): value(value), next(next) {}
    };
    Node* top;
    void deallocate() {
        while (top) {
            Node* next = top->next;
            delete top;
            top = next;
        }
    }
    static Node* copy(Node* node) {
        if (other == nullptr) {
            return nullptr;
        }
        return new Node(node->value, copy(node->next));
    }
public:
    Stack(): top(nullptr) {}
    Stack(const Stack& other): top(copy(other)) {}
};
```

```
Stack& operator=(const Stack& other) {
```

```
    if (this != &other) {
```

```
        deallocate();
```

```
        top = copy(other.top);
```

```
    }
```

```
    return *this;
```

```
}
```

```
~Stack() {
```

```
    deallocate();
```

```
}
```

```
int pop() {
```

```
    if (isEmpty()) {
```

```
        return NAN;
```

```
    }
```

```
    Node* next = top->next;
```

```
    int value = top->value;
```

```
    delete top;
```

```
    top = next;
```

```
    return value;
```

```
}
```

```
void push(int x) {
```

```
    Node* newTop = new Node(x, top);
```

```
    top = newTop;
```

```
}
```

```
bool empty() const {
```

```
    return top == nullptr;
```

```
}
```

```
int peek() const {
```

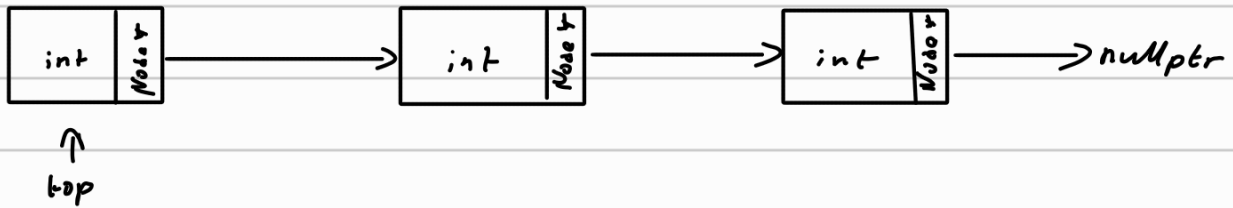
```
    if (isEmpty()) {
```

```
        return NAN;
```

```
    }
```

```
return top->value;
```

```
}
```



- АТД опашка е линейна структура, която има FIFO (First in - First out) организация. Имаме enqueue(), dequeue(), head(), empty()

Реализации

- статична - масив с фиксиран размер. Пазят се индекси first и last, а за да last не излезе от масива $last = (last + 1) \% capacity$ при enqueue и $first = (first + 1) \% capacity$. Има добра локалност
- динамична - динамичен масив. Подобно на статичния пазят first и last, но когато се заняти заглавие нов масив с по-голям размер (например удвоен размер), но нова прави enqueue със сложност $O(n)$ в този случай, но сортизираната сложност е $O(1)$. Има добра локалност
- свързани - едносвързан списък с два указателя, first и last, а операциите са бързи $O(1)$, но няма добра локалност и при enqueue/dequeue правим new/delete

```
class Queue {
```

```
    struct Node {
```

```
        int value;
```

```
        Node* next;
```

```
        Node(int value, Node* next): value(value), next(next) {}
```

```
    };
```

```
    Node* first;
```

```
    Node* last;
```

```

void deallocate() {
    while (First) {
        Node* newFirst = First->next;
        delete First;
        First = newFirst;
    }
    First = nullptr;
    last = nullptr;
}

void copy (Node* start) {
    First = nullptr;
    last = nullptr;
    for (Node* current; current != nullptr; current = current->next) {
        enqueue (Start->value);
    }
}

public:
    Queue(): First(nullptr), last(nullptr) {}
    Queue(const Queue& other): First(nullptr), last(nullptr) {
        copy(other.start)
    }
    Queue& operator=(const Queue& other) {
        if (this != &other) {
            deallocate();
            copy(other->first);
        }
        return *this;
    }
    ~Queue() {
        deallocate();
    }
}

```

```

void enqueue(int x) {
    Node* newNode = new Node(x, nullptr);
    if (isEmpty()) {
        first = newNode;
        last = newNode;
        return;
    }
    last->next = newNode;
    last = newNode;
}

int dequeue() {
    if (isEmpty()) {
        return NAN;
    }
    Node* newFirst = first->next;
    int value = first->value;
    delete first;
    first = newFirst;
    return value;
}

bool isEmpty() const {
    return first == nullptr;
}

int head() const {
    if (isEmpty()) {
        return NAN;
    }
    return first->value;
}

```

Кореново дърво е йерархична структура, а дефиницията е индуктивна:

Кореново дърво е поредна n-орка $(X, T_1, \dots, T_n)$, $n \geq 0$, където X е корен и $T_i, i:1, n$ са коренови поддървета

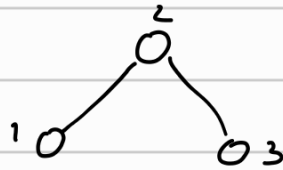
Двоично кореново дърво:

1. $T = \emptyset$ - празно дърво е кореново дърво

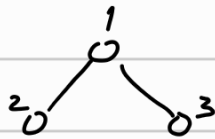
2. $T = (x, L, R)$ е кореново дърво, x - корен, L и R са различни двоични коренови дървета

Одказвачи

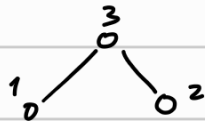
ляво $\rightarrow$ корен $\rightarrow$ дясно



корен $\rightarrow$ ляво $\rightarrow$ дясно



ляво $\rightarrow$ дясно $\rightarrow$ корен



Кореново дърво може да се представя:

1. свързано със списък от указатели към наследници

2. кий-ляво дете и десен sibling - всеки възел носи два указателя - едн към първото си дете и втори към неговия sibling (на неговото ниво)

3. масив от тройки $(data, indexLeft, indexRight)$, а при листа: -1

Двоично дърво

1. Свързано - всеки възел има два указателя: left и right

2. Последовително - коренът е на индекс 0, а дъщеря на i са $2i+1$ и $2i+2$. Последователно е за пълни двоични дървета като например Binary Heap, но едни пълни много малко за дървета които не са пълни или почти пълни

```

class BinTree {
    struct Node {
        int data;
        Node* left;
        Node* right;
        Node(int data, Node* left, Node* right): data(data), left(left), right(right) {}
    };

    Node* root;

    static Node* copy(Node* start) {
        if (start == nullptr) {
            return nullptr;
        }
        return new Node(start->value, copy(start->left), copy(start->right));
    }

    static void deallocate(Node* start) {
        if (start == nullptr) {
            return;
        }
        deallocate(start->left);
        deallocate(start->right);
        delete start;
    }

    static void print(Node* start) {
        if (start == nullptr) {
            return;
        }
        print(start->left);
        std::cout << start->value;
        print(start->right);
    }
}

```


public:

```
BinTree(): root(nullptr) {}
```

```
BinTree(const BinTree& other): root(copy(other.root)) {}
```

```
BinTree& operator=(const BinTree& other) {
```

```
    if (this != &other) {
```

```
        delete root;
```

```
        root = copy(other.root);
```

```
    }
```

```
    return *this;
```

```
}
```

```
~BinTree() {
```

```
    delete root;
```

```
}
```

```
bool isEmpty() const {
```

```
    return root == nullptr;
```

```
}
```

```
void print() const {
```

```
    print(root);
```

```
}
```

```
};
```

Двоично дърво за търсене е двоично кореново дърво, за което във всеки възел (X, L, R) е изпълнено, че X е по-голям от всички елементи в L и по-малък от всички елементи в R . left $\rightarrow$ root $\rightarrow$ right е сортирано

Операции:

- search(x) - търси x някъде от корена ако е по-малък и надясно ако е по-голям
- insert(x) - както при търсене издираме някъде или надясно спрямо корена
- remove - намира се възела и ако
 - е листо - изтрива се

- има едно време както в linked list се минава и знаят работата му
- има две гледища замяната се с най-малкия елемент от лявото поддърво или най-големият от лявото поддърво
- среден случай за insert е $O(\log n)$, защото $n = O(\log n)$
- най-лош случай дървото се е изродило в списък и insert е $O(n)$
- при самобалансирани се дървета, $n = O(\log n)$ и операциите са $O(\log n)$

class BST {

struct Node {

int value;

Node* left;

Node* right;

Node(int value, Node* left, Node* right): value(value), left(left), right(right),

{};

Node* root;

static void deallocate(Node* node) {

if (node == nullptr) {

return;

}

deallocate(node->left);

deallocate(node->right);

delete node;

}

static Node* copy(Node* tree) {

if (tree == nullptr) {

return nullptr;

}

Node* newNode = new Node(tree->value, copy(tree->left), copy(tree->right));

return newNode;

}

```
static void insert(Node*& tree, int x) {
```

```
    if (tree == nullptr) {
```

```
        tree = new Node(x, nullptr, nullptr);
```

```
        return;
```

```
    }
```

```
    if (x < tree->value) {
```

```
        insert(tree->left, x);
```

```
    } else if (x > tree->value) {
```

```
        insert(tree->right, x);
```

```
    }
```

```
}
```

```
static void remove(Node*& node, int x) {
```

```
    if (node == nullptr) {
```

```
        return;
```

```
    }
```

```
    if (x < node->data) {
```

```
        remove(node->left, x);
```

```
    } else if (x > node->data) {
```

```
        remove(node->right, x);
```

```
    } else if (node->left && !node->right) {
```

```
        Node* old = node;
```

```
        node = node->left;
```

```
        delete old;
```

```
    } else if (!node->left && node->right) {
```

```
        Node* old = node;
```

```
        node = node->right;
```

```
        delete old;
```

```
    } else {
```

```
        node->data = extractMin(node->right);
```

```
    }
```

```
}
```

```

static int extractMin(Node*& node) {
    if(node->left) {
        return extractMin(node->left);
    }
    int min = node->value;
    Node* old = node;
    node = node->right;
    delete old;
    return min;
}

```

public:

```

BST(): root(nullptr) {}
BST(const BST& other): root(copy(other.root)) {}
BST& operator=(const BST& other) {
    if(this != &other) {
        deallocate(root);
        root = copy(other.root);
    }
    return *this;
}

```

}

~BST() {

deallocate(root);

root = nullptr;

}

bool search(int x) const {

search(root, x);

}

void insert(int x) {

insert(root, x);

}

```
void remove(int x){  
    remove(root, x);  
}
```